

Standard Operating Procedure (SOP)

System: Node.js / Postgres / Render / Dedicated Disk

Subject: Monthly Technical Maintenance & Security Hardening

I. DATABASE INTEGRITY & PERSISTENCE

1. Postgres Backup Verification

Render performs automated daily backups. However, manual verification is required to prevent silent failure.

- Navigate to **Render Dashboard** > **[DB Name]** > **Backups**.
- Verify the timestamp of the latest snapshot.
- **Manual Snapshot:** Run `pg_dump` via CLI to a secure local directory:

```
pg_dump -h [host] -U [user] -d [db_name] > backup_$(date +%F).sql
```

2. Dedicated Disk (Volume) Management

Your dedicated disk stores persistent data (uploads, sqlite, or logs). Failure occurs at 100% usage.

- Access the Render Shell for the Web Service.
- Check usage: `df -h /mnt/data` (replace with your mount path).
- Audit log sizes: `du -sh /mnt/data/logs/*`.
- **Action:** Rotate logs or archive files older than 90 days.

II. SECURITY & DEPENDENCY AUDITING

1. Node.js Vulnerability Scanning

Dependencies in `package.json` must be audited for CVEs (Common Vulnerabilities and Exposures).

```
# Check for known vulnerabilities
npm audit

# Apply non-breaking security patches
npm audit fix
```

2. GitHub & Environment Hardening

- Review **GitHub Dependabot** alerts and merge security pull requests.
- Ensure `.env` variables are not leaked in logs.
- Check **Render Access Control**: Remove any stale collaborator accounts or API keys.

III. SYSTEM PERFORMANCE & HYGIENE

1. Database Maintenance (Postgres)

To prevent table bloat and optimize the query planner:

```
VACUUM ANALYZE;
```

Monitor "Dead Tuples":

```
SELECT relname, n_dead_tup  
FROM pg_stat_user_tables;
```

2. Log Management (Render)

Render logs are stored in their dashboard, but internal application logs may reside on your disk.

- Check for repeated errors (500 status codes) in the Log Stream.
- Monitor **Memory Utilization** in Render Metrics. If usage exceeds 80% consistently, investigate memory leaks in the Node.js event loop.

CRITICAL: Never run `npm update` without a verified database backup. Major version updates should be performed in a staging environment first.

IV. CHECKLIST SUMMARY

Action Item	Frequency	Command/Tool
Disk Usage Check	Monthly	<code>df -h</code>
NPM Security Audit	Monthly	<code>npm audit</code>
DB Vacuuming	Quarterly	<code>VACUUM ANALYZE</code>
Backup Test Restore	Bi-Annually	<code>pg_restore</code>